Now, when you run this test using `mvn verify`, it should pass, which demonstrates that you now have a working itinerary service. But you aren't done yet. The itinerary service is built on the assumption that the timetable service does its job correctly and uses a "dummy" timetable service (known as a "stub" or "mock" object) to avoid needing a real one. This is a very effective way to build the itinerary service, as you can concentrate exclusively on the business logic of this service. But to get your acceptance criteria, you'll also need to implement this timetable service.

The behavior defined for the mock timetable service provides very precise low-level requirements about how the timetable service should behave. This will be the starting point when you implement this service. You're going to write a new Spock test called `WhenFindingLinesThroughStations.groovy` (once again in the com.bddinaction.chapter2.services package) that builds on these requirements and describes what the timetable service should do in more detail:

```
package com.bddinaction.chapter2.services

import com.bddinaction.chapter2.model.Line
import spock.lang.Specification

class WhenFindingLinesThroughStations extends Specification {

    def timetableService = new TimetableService()

    def "should find the correct lines between two stations"() {
        when:
            def lines = timetableService.findLinesThrough(departure,
                                                          destination)
        then:
            def expectedLine = Line.named(lineName).
                                departingFrom(lineDeparture)
            lines == [expectedLine]
        where:
            departure     | destination   | lineName  | lineDeparture
            "Parramatta"  | "Town Hall"   | "Western" | "Emu Plains"
            "Town Hall"   | "Parramatta"  | "Western" | "North Richmond"
            "Strathfield" | "Epping"      | "Epping"  | "City"
    }
}
```

Action under test.

Timetable service should return these lines.

Use example data from this table. ❶

This test builds on the example from the acceptance criteria to explore the requirements. But unit testing should be more thorough than acceptance testing. In this case, you use a table-driven approach ❶ similar to the JBehave table in listing 2.3. This makes it easy to add a more comprehensive set of examples, which is what you want at this level of detail.

Once you've implemented the `findLinesThrough()` method, you can proceed to the next function you need: finding the arrival times at a given station. Here you can use a similar approach, writing a new Spock specification as your starting point.

EXERCISE 2.1 Write the unit tests for the "Find the arrival times of a given line at a particular station" feature, and implement the corresponding code.